

ICS 321: Database Systems (251) Project # 2

Group: M02

Abdulaziz Alkathiri 202173690

Ibrahim Alshaya 202176470

Information & Computer Science Department, King Fahd

University of Petroleum and Minerals

ICS 321: Database Systems

Sec : 01

Date: 13/12/2025

TOOLS AND RESOURCES USED:

Database: MySQL

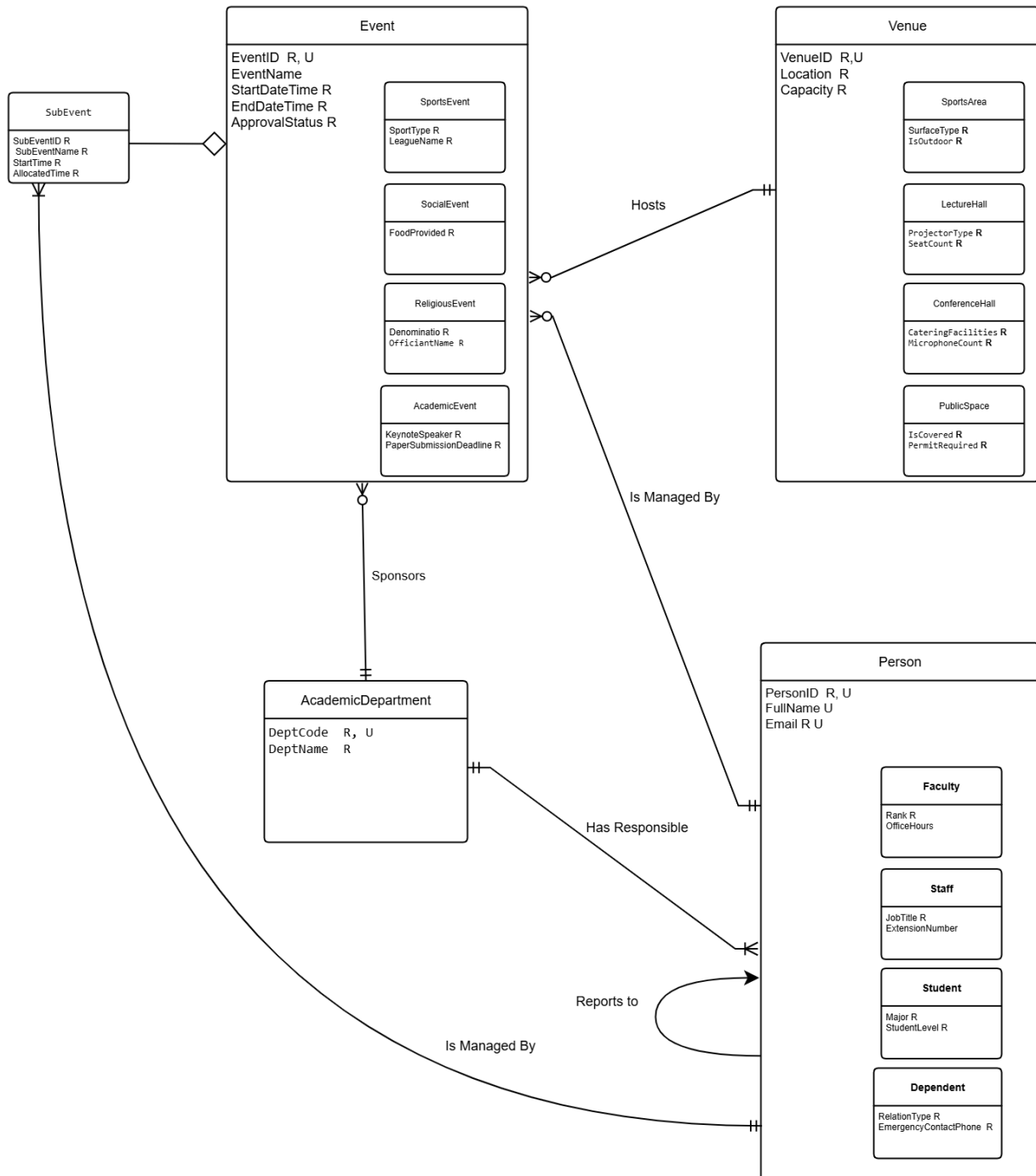
IDE: MySQL workbench

EER Diagram: drow.io

Tasks:

Name	Task1	Task2	Task3
Ibrahim Alshayea	Part A	Part C	Part F
Abdulaziz Alkathiri	Part B	Part D	Part E

Part A - EER diagram:



Part B: LOGICAL SCHEMA AND NORMALIZATION STEPS

I. NORMALIZATION PROCESS

1NF:

Relation: EVENT_REPORT_1NF (

EventID, **SubEventID**, EventName, StartDateTime, EndDateTime, ApprovalStatus,

VenueID, VenueLocation, VenueCapacity,

DeptCode, DeptName,

OrganizerPersonID, OrganizerName, OrganizerEmail,

SubEventName, SubEventStartTime, InChargePersonID

)

Status: In 1NF because every cell has one value and there is a PK.

2NF:

From 1NF, there is 2NF Violation:

- The PK is (EventID + SubEventID).
- "EventName", "VenueID", and "DeptName" depend only on "EventID".
- This violates 2NF.

We Split the table to remove partial dependencies.

Relation 1: EVENT_MASTER_2NF (

EventID, EventName, StartDateTime, VenueID, VenueLocation,

DeptCode, DeptName, OrganizerPersonID, OrganizerName

)

Relation 2: SUB_EVENT (

EventID, **SubEventID**, SubEventName, StartTime, InChargePersonID

)

Now in 2NF. Non-key columns in Relation 1 depend on "EventID".

Non-key columns in Relation 2 depend on "EventID + SubEventID".

BCNF:

Violation in EVENT_MASTER_2NF:

- "VenueLocation" depends on "VenueID". "VenueID" is not unique in this table
- "DeptName" depends on "DeptCode". "DeptCode" is not unique in this table
- "OrganizerName" depends on "OrganizerPersonID". OrganizerPersonID is not unique

We Create separate tables where those determinants (VenueID, DeptCode) are unique.

Final Result:

- Table ACADEMIC_DEPARTMENT (DeptName depends on DeptCode; DeptCode is Unique PK).
- Table VENUE (Location depends on VenueID; VenueID is Unique PK).
- Table PERSON (Name depends on PersonID; PersonID is Unique PK).
- Table EVENT (Removes the dependent columns, keeping only the Keys).

II. FINAL LOGICAL SCHEMA

Notation:

PK = Primary Key (Bold, Unique)

FK = Foreign Key (Italics)

1. STRONG ENTITIES

ACADEMIC_DEPARTMENT (**DeptCode** U , DeptName)

VENUE (**VenueID** U , Location, Capacity)

PERSON (**PersonID** U, FullName, Email)

2. PERSON SUBCLASSES (Organizer Types)

FACULTY (**PersonID** U, Rank, OfficeHours)

FK: PersonID references PERSON(PersonID)

STAFF (**PersonID** U, JobTitle, ExtensionNumber)

FK: PersonID references PERSON(PersonID)

STUDENT (**PersonID** U, Major, StudentLevel)

FK: PersonID references PERSON(PersonID)

DEPENDENT (**PersonID** U, RelationType, EmergencyContactPhone)

FK: PersonID references PERSON(PersonID)

3. VENUE SUBCLASSES

SPORTS_AREA (**VenueID** U, SurfaceType, IsOutdoor)

FK: VenueID references VENUE(VenueID)

LECTURE_HALL (**VenueID** U, ProjectorType, SeatCount)

FK: VenueID references VENUE(VenueID)

CONFERENCE_HALL (**VenueID** U, CateringFacilities, MicrophoneCount)

FK: VenueID references VENUE(VenueID)

PUBLIC_SPACE (**VenueID** U, IsCovered, PermitRequired)

FK: VenueID references VENUE(VenueID)

4. EVENT ENTITY

EVENT (**EventID** U, EventName, StartDateTime, EndDateTime, ApprovalStatus, *VenueID*, *DeptCode*, *OrganizerPersonID*)

FK1: VenueID references VENUE(VenueID)

FK2: DeptCode references ACADEMIC_DEPARTMENT(DeptCode)

FK3: OrganizerPersonID references PERSON(PersonID)

5. EVENT SUBCLASSES

SPORTS_EVENT (**EventID** U, SportType, LeagueName)

FK: EventID references EVENT(EventID)

SOCIAL_EVENT (**EventID** U, FoodProvided)

FK: EventID references EVENT(EventID)

RELIGIOUS_EVENT (**EventID** U, Denomination, OfficiantName)

FK: EventID references EVENT(EventID)

ACADEMIC_EVENT (**EventID** U, KeynoteSpeaker, PaperSubmissionDeadline)

FK: EventID references EVENT(EventID)

6. WEAK ENTITY (Sub-Events)

SUB_EVENT (**EventID** U, **SubEventID** U, SubEventName, StartTime, AllocatedTime, *InChargePersonID*)

PK: Composite Key (EventID + SubEventID)

FK1: EventID references EVENT(EventID)

FK2: InChargePersonID references PERSON(PersonID)

Part C, D, E, and F.

Note: Part F test results are provided in part F

Note: The SQL Statements are provided as SQL files in the zip folder if you want to run them.

PART C:

Start of part C

```
DROP DATABASE IF EXISTS UniversityEventsDB;
```

```
CREATE DATABASE UniversityEventsDB;
```

```
USE UniversityEventsDB;
```

```
-- STRONG ENTITIES
```

```
CREATE TABLE ACADEMIC_DEPARTMENT (
```

```
    DeptCode VARCHAR(10) PRIMARY KEY,
```

```
    DeptName VARCHAR(100) NOT NULL
```

```
);
```

```
CREATE TABLE VENUE (
```

```
    VenueID INT PRIMARY KEY AUTO_INCREMENT,
```

```
    Location VARCHAR(255) NOT NULL,
```

```
    Capacity INT
```

```
);
```

```
CREATE TABLE PERSON (
```

```
    PersonID INT PRIMARY KEY AUTO_INCREMENT,
```

```
    FullName VARCHAR(255) NOT NULL,
```

```
    Email VARCHAR(255) UNIQUE NOT NULL
```

```
);
```

```
-- PERSON SUBCLASSES
```

```
CREATE TABLE FACULTY (
```



```
    PersonID INT PRIMARY KEY,  
  
    RankTitle VARCHAR(50),  
  
    OfficeHours VARCHAR(100),  
  
    FOREIGN KEY (PersonID) REFERENCES PERSON(PersonID) ON DELETE CASCADE  
);
```

```
CREATE TABLE STAFF (  
  
    PersonID INT PRIMARY KEY,  
  
    JobTitle VARCHAR(100),  
  
    ExtensionNumber VARCHAR(20),  
  
    FOREIGN KEY (PersonID) REFERENCES PERSON(PersonID) ON DELETE CASCADE  
);
```

```
CREATE TABLE STUDENT (  
  
    PersonID INT PRIMARY KEY,  
  
    Major VARCHAR(50),  
  
    StudentLevel VARCHAR(20),  
  
    FOREIGN KEY (PersonID) REFERENCES PERSON(PersonID) ON DELETE CASCADE  
);
```

```
CREATE TABLE DEPENDENT (  
  
    PersonID INT PRIMARY KEY,  
  
    RelationType VARCHAR(50),  
  
    EmergencyContactPhone VARCHAR(20),  
  
    FOREIGN KEY (PersonID) REFERENCES PERSON(PersonID) ON DELETE CASCADE  
);
```

-- VENUE SUBCLASSES

```
CREATE TABLE SPORTS_AREA (  
  
    VenueID INT PRIMARY KEY,  
  
    SurfaceType VARCHAR(50),  
  
    IsOutdoor BOOLEAN,  
  
    FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID) ON DELETE CASCADE  
);
```

```
CREATE TABLE LECTURE_HALL (  
    VenueID INT PRIMARY KEY,  
    ProjectorType VARCHAR(50),  
    SeatCount INT,  
    FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID) ON DELETE CASCADE  
);
```

```
CREATE TABLE CONFERENCE_HALL (  
    VenueID INT PRIMARY KEY,  
    CateringFacilities BOOLEAN,  
    MicrophoneCount INT,  
    FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID) ON DELETE CASCADE  
);
```

```
CREATE TABLE PUBLIC_SPACE (  
    VenueID INT PRIMARY KEY,  
    IsCovered BOOLEAN,  
    PermitRequired BOOLEAN,  
    FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID) ON DELETE CASCADE  
);
```

-- EVENT ENTITY

```
CREATE TABLE EVENT (  
    EventID INT PRIMARY KEY AUTO_INCREMENT,  
    EventName VARCHAR(200) NOT NULL,  
    StartDateTime DATETIME NOT NULL,  
    EndDateTime DATETIME NOT NULL,  
    ApprovalStatus ENUM('Pending', 'Approved', 'Rejected', 'Cancelled') DEFAULT 'Pending',
```

-- Req 7

```
RejectionJustification VARCHAR(255),
```

```
VenueID INT,
```

DeptCode VARCHAR(10),

OrganizerPersonID INT,

FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID),

FOREIGN KEY (DeptCode) REFERENCES ACADEMIC_DEPARTMENT(DeptCode),

FOREIGN KEY (OrganizerPersonID) REFERENCES PERSON(PersonID),

-- Req 9

CONSTRAINT Chk_Duration CHECK (DATEDIFF(EndTime, StartTime) <= 3),

CONSTRAINT Chk_StartTime CHECK (CAST(StartTime AS TIME) >= '08:00:00'),

CONSTRAINT Chk_EndTime CHECK (CAST(EndTime AS TIME) <= '23:59:59')

);

-- EVENT SUBCLASSES

CREATE TABLE SPORTS_EVENT (

EventID INT PRIMARY KEY,

SportType VARCHAR(50),

LeagueName VARCHAR(100),

FOREIGN KEY (EventID) REFERENCES EVENT(EventID) ON DELETE CASCADE

);

CREATE TABLE SOCIAL_EVENT (

EventID INT PRIMARY KEY,

FoodProvided BOOLEAN,

FOREIGN KEY (EventID) REFERENCES EVENT(EventID) ON DELETE CASCADE

);

CREATE TABLE RELIGIOUS_EVENT (

EventID INT PRIMARY KEY,

Denomination VARCHAR(50),

OfficiantName VARCHAR(100),

FOREIGN KEY (EventID) REFERENCES EVENT(EventID) ON DELETE CASCADE

);

```
CREATE TABLE ACADEMIC_EVENT (  
    EventID INT PRIMARY KEY,  
    KeynoteSpeaker VARCHAR(100),  
    PaperSubmissionDeadline DATETIME,  
    FOREIGN KEY (EventID) REFERENCES EVENT(EventID) ON DELETE CASCADE  
);
```

-- WEAK ENTITY

```
CREATE TABLE SUB_EVENT (  
    EventID INT,  
    SubEventID INT,  
    SubEventName VARCHAR(150),  
    StartTime DATETIME,  
    AllocatedTime VARCHAR(50),  
    InChargePersonID INT,  
  
    PRIMARY KEY (EventID, SubEventID),  
    FOREIGN KEY (EventID) REFERENCES EVENT(EventID) ON DELETE CASCADE,  
    FOREIGN KEY (InChargePersonID) REFERENCES PERSON(PersonID)  
);
```

-- VIEW

```
CREATE VIEW ApprovedEventDetails AS  
SELECT  
    E.EventName,  
    E.StartDateTime,  
    V.Location,  
    P.FullName AS Organizer,  
    D.DeptName  
FROM EVENT E  
JOIN VENUE V ON E.VenueID = V.VenueID  
JOIN PERSON P ON E.OrganizerPersonID = P.PersonID  
JOIN ACADEMIC_DEPARTMENT D ON E.DeptCode = D.DeptCode  
WHERE E.ApprovalStatus = 'Approved';
```

-- Req 8

```
CREATE TABLE EMAIL_LOG (  
  
    LogID INT AUTO_INCREMENT PRIMARY KEY,  
  
    EventID INT,  
  
    Message VARCHAR(255),  
  
    SentDate DATETIME DEFAULT CURRENT_TIMESTAMP  
  
);
```

DELIMITER //

```
CREATE TRIGGER trg_EventApproved  
  
AFTER UPDATE ON EVENT  
  
FOR EACH ROW  
  
BEGIN  
  
    IF NEW.ApprovalStatus = 'Approved' AND OLD.ApprovalStatus != 'Approved' THEN  
  
        INSERT INTO EMAIL_LOG (EventID, Message)  
  
        VALUES (NEW.EventID, 'Email sent to Maintenance, Office Services, Security: Event Approved.');  
    END IF;  
  
END //
```

DELIMITER ;

End of part C

Part D:

Start of Part D

```
USE UniversityEventsDB;
```

```
-- CLEAR OLD DATA
```

```
SET FOREIGN_KEY_CHECKS = 0;
```

```
TRUNCATE TABLE SUB_EVENT;
```

```
TRUNCATE TABLE ACADEMIC_EVENT;
```

```
TRUNCATE TABLE RELIGIOUS_EVENT;
```

```
TRUNCATE TABLE SOCIAL_EVENT;
```

```
TRUNCATE TABLE SPORTS_EVENT;
```

```
TRUNCATE TABLE EVENT;
```

```
TRUNCATE TABLE PUBLIC_SPACE;
```

```
TRUNCATE TABLE CONFERENCE_HALL;
```

```
TRUNCATE TABLE LECTURE_HALL;
```

```
TRUNCATE TABLE SPORTS_AREA;
```

```
TRUNCATE TABLE DEPENDENT;
```

```
TRUNCATE TABLE STUDENT;
```

```
TRUNCATE TABLE STAFF;
```

```
TRUNCATE TABLE FACULTY;
```

```
TRUNCATE TABLE PERSON;
```

```
TRUNCATE TABLE VENUE;
```

```
TRUNCATE TABLE ACADEMIC_DEPARTMENT;
```

```
TRUNCATE TABLE EMAIL_LOG;
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

-- INSERT STRONG ENTITIES

-- ACADEMIC_DEPARTMENT

INSERT INTO ACADEMIC_DEPARTMENT (DeptCode, DeptName) VALUES

('CS', 'Computer Science'),

('MATH', 'Mathematics'),

('PHYS', 'Physics'),

('ENG', 'English Literature'),

('BIO', 'Biology');

-- VENUE

INSERT INTO VENUE (VenueID, Location, Capacity) VALUES

(101, 'North Field', 5000), (102, 'East Gym', 200), (103, 'Tennis Court A', 50), (104, 'Swimming Pool', 100), (105, 'Soccer Pitch', 3000),

(201, 'Science Hall A', 300), (202, 'Main Auditorium', 1000), (203, 'Math Room 101', 50), (204, 'Bio Lab 3', 40), (205, 'Eng Room 202', 60),

(301, 'Grand Conference', 500), (302, 'Meeting Room A', 20), (303, 'Meeting Room B', 20), (304, 'Board Room', 15), (305, 'Banquet Hall', 200),

(401, 'Central Plaza', 10000), (402, 'Library Lawn', 500), (403, 'Student Center Lobby', 100), (404, 'Main Gate', 50), (405, 'Rooftop Garden', 80);

-- PERSON

INSERT INTO PERSON (PersonID, FullName, Email) VALUES

-- Faculty

(1, 'Dr. Alice Smith', 'alice@uni.edu'), (2, 'Prof. Bob Jones', 'bob@uni.edu'), (3, 'Dr. Carol White', 'carol@uni.edu'), (4, 'Prof. Dave Black', 'dave@uni.edu'), (5, 'Dr. Eve Green', 'eve@uni.edu'),

-- Staff

(6, 'Frank Admin', 'frank@uni.edu'), (7, 'Grace Sec', 'grace@uni.edu'), (8, 'Hank Tech', 'hank@uni.edu'), (9, 'Ivy Maintenance', 'ivy@uni.edu'), (10, 'Jack Janitor', 'jack@uni.edu'),

-- Students

(11, 'Kevin Kid', 'kevin@student.edu'), (12, 'Laura Learner', 'laura@student.edu'), (13, 'Mike Major', 'mike@student.edu'), (14, 'Nancy New', 'nancy@student.edu'), (15, 'Oscar Old', 'oscar@student.edu'),

-- Dependents

(16, 'Penny Smith', 'penny@gmail.com'), (17, 'Quinn Jones', 'quinn@gmail.com'), (18, 'Randy White', 'randy@gmail.com'), (19, 'Sarah Black', 'sarah@gmail.com'), (20, 'Tim Green', 'tim@gmail.com');

-- INSERT SUBCLASS DATA

-- FACULTY

INSERT INTO FACULTY (PersonID, RankTitle, OfficeHours) VALUES

(1, 'Professor', 'Mon 10-12'), (2, 'Associate Prof', 'Tue 2-4'), (3, 'Lecturer', 'Wed 9-11'), (4, 'Professor', 'Thu 1-3'), (5, 'Assistant Prof', 'Fri 10-12');

-- STAFF

INSERT INTO STAFF (PersonID, JobTitle, ExtensionNumber) VALUES

(6, 'Administrator', 'x101'), (7, 'Secretary', 'x102'), (8, 'Technician', 'x103'), (9, 'Manager', 'x104'), (10, 'Coordinator', 'x105');

-- STUDENT

INSERT INTO STUDENT (PersonID, Major, StudentLevel) VALUES

(11, 'CS', 'Senior'), (12, 'Math', 'Junior'), (13, 'Physics', 'Sophomore'), (14, 'Biology', 'Freshman'), (15, 'English', 'Senior');

-- DEPENDENT

INSERT INTO DEPENDENT (PersonID, RelationType, EmergencyContactPhone) VALUES

(16, 'Daughter', '555-0101'), (17, 'Son', '555-0102'), (18, 'Spouse', '555-0103'), (19, 'Child', '555-0104'), (20, 'Spouse', '555-0105');

-- INSERT VENUE SUBCLASS DATA

INSERT INTO SPORTS_AREA (VenueID, SurfaceType, IsOutdoor) VALUES

(101, 'Grass', TRUE), (102, 'Hardwood', FALSE), (103, 'Clay', TRUE), (104, 'Water', FALSE), (105, 'Turf', TRUE);

INSERT INTO LECTURE_HALL (VenueID, ProjectorType, SeatCount) VALUES

(201, '4K Laser', 300), (202, 'Standard', 1000), (203, 'None', 50), (204, 'SmartBoard', 40), (205, 'Standard', 60);

INSERT INTO CONFERENCE_HALL (VenueID, CateringFacilities, MicrophoneCount) VALUES

(301, TRUE, 10), (302, FALSE, 1), (303, FALSE, 1), (304, TRUE, 5), (305, TRUE, 4);

INSERT INTO PUBLIC_SPACE (VenueID, IsCovered, PermitRequired) VALUES

(401, FALSE, TRUE), (402, FALSE, FALSE), (403, TRUE, FALSE), (404, FALSE, TRUE), (405, TRUE, TRUE);

-- INSERT MAIN EVENTS

INSERT INTO EVENT (EventID, EventName, StartDateTime, EndDateTime, ApprovalStatus, RejectionJustification, VenueID, DeptCode, OrganizerPersonID) VALUES

-- Sports Events

(1, 'Varsity Soccer Game', '2023-11-01 14:00:00', '2023-11-01 16:00:00', 'Approved', NULL, 101, 'BIO', 1),
(2, 'Swim Meet', '2023-11-02 09:00:00', '2023-11-02 12:00:00', 'Pending', NULL, 104, 'BIO', 2),
(3, 'Tennis Finals', '2023-11-03 10:00:00', '2023-11-03 13:00:00', 'Approved', NULL, 103, 'MATH', 11),
(4, 'Basketball Practice', '2023-11-04 18:00:00', '2023-11-04 20:00:00', 'Rejected', 'Conflicting Schedule', 102, 'CS', 12),
(5, 'Charity Run', '2023-11-05 08:00:00', '2023-11-05 11:00:00', 'Approved', NULL, 401, 'PHYS', 6),

-- Social Events

(6, 'Welcome Picnic', '2023-12-01 12:00:00', '2023-12-01 15:00:00', 'Approved', NULL, 402, 'ENG', 13),
(7, 'Movie Night', '2023-12-02 19:00:00', '2023-12-02 22:00:00', 'Approved', NULL, 201, 'CS', 14),
(8, 'Staff Dinner', '2023-12-03 18:00:00', '2023-12-03 21:00:00', 'Pending', NULL, 305, 'MATH', 7),
(9, 'Alumni Mixer', '2023-12-04 17:00:00', '2023-12-04 19:00:00', 'Approved', NULL, 301, 'BIO', 3),
(10, 'Coffee Hour', '2023-12-05 10:00:00', '2023-12-05 11:00:00', 'Approved', NULL, 403, 'ENG', 15),

-- Religious Events

(11, 'Morning Prayer', '2023-10-10 08:00:00', '2023-10-10 09:00:00', 'Approved', NULL, 405, 'ENG', 4),
(12, 'Holiday Service', '2023-10-11 18:00:00', '2023-10-11 20:00:00', 'Approved', NULL, 202, 'PHYS', 5),
(13, 'Meditation', '2023-10-12 12:00:00', '2023-10-12 13:00:00', 'Pending', NULL, 402, 'BIO', 8),
(14, 'Study Circle', '2023-10-13 14:00:00', '2023-10-13 16:00:00', 'Approved', NULL, 302, 'MATH', 9),
(15, 'Choir Practice', '2023-10-14 17:00:00', '2023-10-14 19:00:00', 'Rejected', 'Noise complaints', 205, 'CS', 10),

-- Academic Events

(16, 'Guest Lecture: AI', '2023-09-01 14:00:00', '2023-09-01 16:00:00', 'Approved', NULL, 201, 'CS', 1),
(17, 'Math Symposium', '2023-09-02 09:00:00', '2023-09-02 17:00:00', 'Approved', NULL, 301, 'MATH', 2),
(18, 'Physics Workshop', '2023-09-03 10:00:00', '2023-09-03 12:00:00', 'Pending', NULL, 203, 'PHYS', 3),
(19, 'Bio Research Fair', '2023-09-04 11:00:00', '2023-09-04 15:00:00', 'Approved', NULL, 403, 'BIO', 4),
(20, 'Literature Debate', '2023-09-05 15:00:00', '2023-09-05 17:00:00', 'Approved', NULL, 205, 'ENG', 5);

-- INSERT EVENT SUBCLASSES

INSERT INTO SPORTS_EVENT (EventID, SportType, LeagueName) VALUES

(1, 'Soccer', 'Varsity League'), (2, 'Swimming', 'Regional'), (3, 'Tennis', 'Intramural'), (4, 'Basketball', 'Training'), (5, 'Running', 'Charity');

INSERT INTO SOCIAL_EVENT (EventID, FoodProvided) VALUES

```
(6, TRUE), (7, TRUE), (8, TRUE), (9, TRUE), (10, FALSE);
```

```
INSERT INTO RELIGIOUS_EVENT (EventID, Denomination, OfficiantName) VALUES
```

```
(11, 'Interfaith', 'Rev. Green'), (12, 'Christian', 'Pastor Brown'), (13, 'Buddhist', 'Monk Li'), (14, 'Muslim', 'Imam Ahmed'), (15, 'Catholic',  
'Father John');
```

```
INSERT INTO ACADEMIC_EVENT (EventID, KeynoteSpeaker, PaperSubmissionDeadline) VALUES
```

```
(16, 'Elon Musk', '2023-08-01 23:59:59'), (17, 'Terence Tao', '2023-08-02 23:59:59'), (18, 'Neil deGrasse Tyson', NULL), (19, 'Jane Goodall',  
'2023-08-04 23:59:59'), (20, 'Margaret Atwood', NULL);
```

```
-- INSERT WEAK ENTITY (Sub-Events)
```

```
-- Event 16 has sub-events
```

```
INSERT INTO SUB_EVENT (EventID, SubEventID, SubEventName, StartTime, AllocatedTime, InChargePersonID) VALUES
```

```
(16, 1, 'Introduction', '2023-09-01 14:00:00', '15 mins', 11),
```

```
(16, 2, 'Main Talk', '2023-09-01 14:15:00', '60 mins', 11),
```

```
(16, 3, 'Q&A Session', '2023-09-01 15:15:00', '30 mins', 12),
```

```
(16, 4, 'Networking', '2023-09-01 15:45:00', '15 mins', 13),
```

```
(16, 5, 'Closing', '2023-09-01 16:00:00', '10 mins', 14);
```

End of part D

Part E:

Start of part E

USE UniversityEventsDB;

DROP PROCEDURE IF EXISTS Project2_Reset_and_Populate;

-- DEFINE THE PROCEDURE

DELIMITER //

CREATE PROCEDURE Project2_Reset_and_Populate()

BEGIN

-- DISABLE FOREIGN KEYS (To allow dropping tables freely)

SET FOREIGN_KEY_CHECKS = 0;

DROP TABLE IF EXISTS SUB_EVENT, ACADEMIC_EVENT, RELIGIOUS_EVENT, SOCIAL_EVENT, SPORTS_EVENT, EVENT;

DROP TABLE IF EXISTS PUBLIC_SPACE, CONFERENCE_HALL, LECTURE_HALL, SPORTS_AREA, VENUE;

DROP TABLE IF EXISTS DEPENDENT, STUDENT, STAFF, FACULTY, PERSON, ACADEMIC_DEPARTMENT;

DROP TABLE IF EXISTS EMAIL_LOG;

-- C. CREATE TABLES

CREATE TABLE ACADEMIC_DEPARTMENT (

DeptCode VARCHAR(10) PRIMARY KEY,

DeptName VARCHAR(100) NOT NULL

);

```
CREATE TABLE VENUE (  
    VenueID INT PRIMARY KEY AUTO_INCREMENT,  
    Location VARCHAR(255) NOT NULL,  
    Capacity INT  
);
```

```
CREATE TABLE PERSON (  
    PersonID INT PRIMARY KEY AUTO_INCREMENT,  
    FullName VARCHAR(255) NOT NULL,  
    Email VARCHAR(255) UNIQUE NOT NULL  
);
```

```
CREATE TABLE FACULTY (  
    PersonID INT PRIMARY KEY, RankTitle VARCHAR(50), OfficeHours VARCHAR(100),  
    FOREIGN KEY (PersonID) REFERENCES PERSON(PersonID) ON DELETE CASCADE  
);
```

```
CREATE TABLE STAFF (  
    PersonID INT PRIMARY KEY, JobTitle VARCHAR(100), ExtensionNumber VARCHAR(20),  
    FOREIGN KEY (PersonID) REFERENCES PERSON(PersonID) ON DELETE CASCADE  
);
```

```
CREATE TABLE STUDENT (  
    PersonID INT PRIMARY KEY, Major VARCHAR(50), StudentLevel VARCHAR(20),  
    FOREIGN KEY (PersonID) REFERENCES PERSON(PersonID) ON DELETE CASCADE  
);
```

```
CREATE TABLE DEPENDENT (  
    PersonID INT PRIMARY KEY, RelationType VARCHAR(50), EmergencyContactPhone VARCHAR(20),  
    FOREIGN KEY (PersonID) REFERENCES PERSON(PersonID) ON DELETE CASCADE  
);
```

```
CREATE TABLE SPORTS_AREA (  
    PersonID INT PRIMARY KEY, SportArea VARCHAR(50),  
    FOREIGN KEY (PersonID) REFERENCES PERSON(PersonID) ON DELETE CASCADE  
);
```

```
VenueID INT PRIMARY KEY, SurfaceType VARCHAR(50), IsOutdoor BOOLEAN,  
FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID) ON DELETE CASCADE  
);
```

```
CREATE TABLE LECTURE_HALL (  
VenueID INT PRIMARY KEY, ProjectorType VARCHAR(50), SeatCount INT,  
FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID) ON DELETE CASCADE  
);
```

```
CREATE TABLE CONFERENCE_HALL (  
VenueID INT PRIMARY KEY, CateringFacilities BOOLEAN, MicrophoneCount INT,  
FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID) ON DELETE CASCADE  
);
```

```
CREATE TABLE PUBLIC_SPACE (  
VenueID INT PRIMARY KEY, IsCovered BOOLEAN, PermitRequired BOOLEAN,  
FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID) ON DELETE CASCADE  
);
```

```
CREATE TABLE EVENT (  
EventID INT PRIMARY KEY AUTO_INCREMENT,  
EventName VARCHAR(200) NOT NULL,  
StartDateTime DATETIME NOT NULL,  
EndDateTime DATETIME NOT NULL,  
ApprovalStatus ENUM('Pending', 'Approved', 'Rejected', 'Cancelled') DEFAULT 'Pending',  
RejectionJustification VARCHAR(255),  
VenueID INT, DeptCode VARCHAR(10), OrganizerPersonID INT,  
FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID),  
FOREIGN KEY (DeptCode) REFERENCES ACADEMIC_DEPARTMENT(DeptCode),  
FOREIGN KEY (OrganizerPersonID) REFERENCES PERSON(PersonID),  
CONSTRAINT Chk_Duration CHECK (DATEDIFF(EndDateTime, StartDateTime) <= 3),  
CONSTRAINT Chk_StartTime CHECK (CAST(StartDateTime AS TIME) >= '08:00:00'),  
CONSTRAINT Chk_EndTime CHECK (CAST(EndDateTime AS TIME) <= '23:59:59')  
);
```

```
CREATE TABLE SPORTS_EVENT (
```

```
    EventID INT PRIMARY KEY, SportType VARCHAR(50), LeagueName VARCHAR(100),
```

```
    FOREIGN KEY (EventID) REFERENCES EVENT(EventID) ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE SOCIAL_EVENT (
```

```
    EventID INT PRIMARY KEY, FoodProvided BOOLEAN,
```

```
    FOREIGN KEY (EventID) REFERENCES EVENT(EventID) ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE RELIGIOUS_EVENT (
```

```
    EventID INT PRIMARY KEY, Denomination VARCHAR(50), OfficiantName VARCHAR(100),
```

```
    FOREIGN KEY (EventID) REFERENCES EVENT(EventID) ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE ACADEMIC_EVENT (
```

```
    EventID INT PRIMARY KEY, KeynoteSpeaker VARCHAR(100), PaperSubmissionDeadline DATETIME,
```

```
    FOREIGN KEY (EventID) REFERENCES EVENT(EventID) ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE SUB_EVENT (
```

```
    EventID INT, SubEventID INT, SubEventName VARCHAR(150), StartTime DATETIME, AllocatedTime VARCHAR(50), InChargePersonID  
INT,
```

```
    PRIMARY KEY (EventID, SubEventID),
```

```
    FOREIGN KEY (EventID) REFERENCES EVENT(EventID) ON DELETE CASCADE,
```

```
    FOREIGN KEY (InChargePersonID) REFERENCES PERSON(PersonID)
```

```
);
```

```
CREATE TABLE EMAIL_LOG (
```

```
    LogID INT AUTO_INCREMENT PRIMARY KEY, EventID INT, Message VARCHAR(255), SentDate DATETIME DEFAULT  
CURRENT_TIMESTAMP
```

```
);
```

-- D. POPULATE DATA (Using data from Part D)

INSERT INTO ACADEMIC_DEPARTMENT VALUES

('CS', 'Computer Science'), ('MATH', 'Mathematics'), ('PHYS', 'Physics'), ('ENG', 'English Literature'), ('BIO', 'Biology');

INSERT INTO VENUE VALUES

(101, 'North Field', 5000), (102, 'East Gym', 200), (103, 'Tennis Court A', 50), (104, 'Swimming Pool', 100), (105, 'Soccer Pitch', 3000),
(201, 'Science Hall A', 300), (202, 'Main Auditorium', 1000), (203, 'Math Room 101', 50), (204, 'Bio Lab 3', 40), (205, 'Eng Room 202', 60),
(301, 'Grand Conference', 500), (302, 'Meeting Room A', 20), (303, 'Meeting Room B', 20), (304, 'Board Room', 15), (305, 'Banquet Hall',
200),
(401, 'Central Plaza', 10000), (402, 'Library Lawn', 500), (403, 'Student Center Lobby', 100), (404, 'Main Gate', 50), (405, 'Rooftop Garden',
80);

INSERT INTO PERSON VALUES

(1, 'Dr. Alice Smith', 'alice@uni.edu'), (2, 'Prof. Bob Jones', 'bob@uni.edu'), (3, 'Dr. Carol White', 'carol@uni.edu'), (4, 'Prof. Dave Black',
'dave@uni.edu'), (5, 'Dr. Eve Green', 'eve@uni.edu'),
(6, 'Frank Admin', 'frank@uni.edu'), (7, 'Grace Sec', 'grace@uni.edu'), (8, 'Hank Tech', 'hank@uni.edu'), (9, 'Ivy Maintenance',
'ivy@uni.edu'), (10, 'Jack Janitor', 'jack@uni.edu'),
(11, 'Kevin Kid', 'kevin@student.edu'), (12, 'Laura Learner', 'laura@student.edu'), (13, 'Mike Major', 'mike@student.edu'), (14, 'Nancy
New', 'nancy@student.edu'), (15, 'Oscar Old', 'oscar@student.edu'),
(16, 'Penny Smith', 'penny@gmail.com'), (17, 'Quinn Jones', 'quinn@gmail.com'), (18, 'Randy White', 'randy@gmail.com'), (19, 'Sarah
Black', 'sarah@gmail.com'), (20, 'Tim Green', 'tim@gmail.com');

INSERT INTO FACULTY VALUES (1, 'Professor', 'Mon 10-12'), (2, 'Associate Prof', 'Tue 2-4'), (3, 'Lecturer', 'Wed 9-11'), (4, 'Professor', 'Thu
1-3'), (5, 'Assistant Prof', 'Fri 10-12');

INSERT INTO STAFF VALUES (6, 'Administrator', 'x101'), (7, 'Secretary', 'x102'), (8, 'Technician', 'x103'), (9, 'Manager', 'x104'), (10,
'Coordinator', 'x105');

INSERT INTO STUDENT VALUES (11, 'CS', 'Senior'), (12, 'Math', 'Junior'), (13, 'Physics', 'Sophomore'), (14, 'Biology', 'Freshman'), (15,
'English', 'Senior');

INSERT INTO DEPENDENT VALUES (16, 'Daughter', '555-0101'), (17, 'Son', '555-0102'), (18, 'Spouse', '555-0103'), (19, 'Child', '555-
0104'), (20, 'Spouse', '555-0105');

INSERT INTO SPORTS_AREA VALUES (101, 'Grass', TRUE), (102, 'Hardwood', FALSE), (103, 'Clay', TRUE), (104, 'Water', FALSE), (105,
'Turf', TRUE);

INSERT INTO LECTURE_HALL VALUES (201, '4K Laser', 300), (202, 'Standard', 1000), (203, 'None', 50), (204, 'SmartBoard', 40), (205,
'Standard', 60);

INSERT INTO CONFERENCE_HALL VALUES (301, TRUE, 10), (302, FALSE, 1), (303, FALSE, 1), (304, TRUE, 5), (305, TRUE, 4);

INSERT INTO PUBLIC_SPACE VALUES (401, FALSE, TRUE), (402, FALSE, FALSE), (403, TRUE, FALSE), (404, FALSE, TRUE), (405, TRUE,
TRUE);

INSERT INTO EVENT VALUES

(1, 'Varsity Soccer Game', '2023-11-01 14:00:00', '2023-11-01 16:00:00', 'Approved', NULL, 101, 'BIO', 1),
(2, 'Swim Meet', '2023-11-02 09:00:00', '2023-11-02 12:00:00', 'Pending', NULL, 104, 'BIO', 2),
(3, 'Tennis Finals', '2023-11-03 10:00:00', '2023-11-03 13:00:00', 'Approved', NULL, 103, 'MATH', 11),
(4, 'Basketball Practice', '2023-11-04 18:00:00', '2023-11-04 20:00:00', 'Rejected', 'Conflicting Schedule', 102, 'CS', 12),
(5, 'Charity Run', '2023-11-05 08:00:00', '2023-11-05 11:00:00', 'Approved', NULL, 401, 'PHYS', 6),
(6, 'Welcome Picnic', '2023-12-01 12:00:00', '2023-12-01 15:00:00', 'Approved', NULL, 402, 'ENG', 13),
(7, 'Movie Night', '2023-12-02 19:00:00', '2023-12-02 22:00:00', 'Approved', NULL, 201, 'CS', 14),
(8, 'Staff Dinner', '2023-12-03 18:00:00', '2023-12-03 21:00:00', 'Pending', NULL, 305, 'MATH', 7),
(9, 'Alumni Mixer', '2023-12-04 17:00:00', '2023-12-04 19:00:00', 'Approved', NULL, 301, 'BIO', 3),
(10, 'Coffee Hour', '2023-12-05 10:00:00', '2023-12-05 11:00:00', 'Approved', NULL, 403, 'ENG', 15),
(11, 'Morning Prayer', '2023-10-10 08:00:00', '2023-10-10 09:00:00', 'Approved', NULL, 405, 'ENG', 4),
(12, 'Holiday Service', '2023-10-11 18:00:00', '2023-10-11 20:00:00', 'Approved', NULL, 202, 'PHYS', 5),
(13, 'Meditation', '2023-10-12 12:00:00', '2023-10-12 13:00:00', 'Pending', NULL, 402, 'BIO', 8),
(14, 'Study Circle', '2023-10-13 14:00:00', '2023-10-13 16:00:00', 'Approved', NULL, 302, 'MATH', 9),
(15, 'Choir Practice', '2023-10-14 17:00:00', '2023-10-14 19:00:00', 'Rejected', 'Noise complaints', 205, 'CS', 10),
(16, 'Guest Lecture: AI', '2023-09-01 14:00:00', '2023-09-01 16:00:00', 'Approved', NULL, 201, 'CS', 1),
(17, 'Math Symposium', '2023-09-02 09:00:00', '2023-09-02 17:00:00', 'Approved', NULL, 301, 'MATH', 2),
(18, 'Physics Workshop', '2023-09-03 10:00:00', '2023-09-03 12:00:00', 'Pending', NULL, 203, 'PHYS', 3),
(19, 'Bio Research Fair', '2023-09-04 11:00:00', '2023-09-04 15:00:00', 'Approved', NULL, 403, 'BIO', 4),
(20, 'Literature Debate', '2023-09-05 15:00:00', '2023-09-05 17:00:00', 'Approved', NULL, 205, 'ENG', 5);

INSERT INTO SPORTS_EVENT VALUES (1, 'Soccer', 'Varsity League'), (2, 'Swimming', 'Regional'), (3, 'Tennis', 'Intramural'), (4, 'Basketball', 'Training'), (5, 'Running', 'Charity');

INSERT INTO SOCIAL_EVENT VALUES (6, TRUE), (7, TRUE), (8, TRUE), (9, TRUE), (10, FALSE);

INSERT INTO RELIGIOUS_EVENT VALUES (11, 'Interfaith', 'Rev. Green'), (12, 'Christian', 'Pastor Brown'), (13, 'Buddhist', 'Monk Li'), (14, 'Muslim', 'Imam Ahmed'), (15, 'Catholic', 'Father John');

INSERT INTO ACADEMIC_EVENT VALUES (16, 'Elon Musk', '2023-08-01 23:59:59'), (17, 'Terence Tao', '2023-08-02 23:59:59'), (18, 'Neil deGrasse Tyson', NULL), (19, 'Jane Goodall', '2023-08-04 23:59:59'), (20, 'Margaret Atwood', NULL);

INSERT INTO SUB_EVENT VALUES

(16, 1, 'Introduction', '2023-09-01 14:00:00', '15 mins', 11),
(16, 2, 'Main Talk', '2023-09-01 14:15:00', '60 mins', 11),
(16, 3, 'Q&A Session', '2023-09-01 15:15:00', '30 mins', 12),
(16, 4, 'Networking', '2023-09-01 15:45:00', '15 mins', 13),


```
(16, 5, 'Closing', '2023-09-01 16:00:00', '10 mins', 14);
```

```
-- E. RE-ENABLE FOREIGN KEYS
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

```
END //
```

```
DELIMITER ;
```

```
-- EXECUTE PROCEDURE
```

```
CALL Project2_Reset_and_Populate();
```

```
-- RE-CREATE TRIGGERS AND VIEWS
```

```
DROP VIEW IF EXISTS ApprovedEventDetails;
```

```
CREATE VIEW ApprovedEventDetails AS
```

```
SELECT E.EventName, E.StartDateTime, V.Location, P.FullName AS Organizer, D.DeptName
```

```
FROM EVENT E
```

```
JOIN VENUE V ON E.VenueID = V.VenueID
```

```
JOIN PERSON P ON E.OrganizerPersonID = P.PersonID
```

```
JOIN ACADEMIC_DEPARTMENT D ON E.DeptCode = D.DeptCode
```

```
WHERE E.ApprovalStatus = 'Approved';
```

```
DROP TRIGGER IF EXISTS trg_EventApproved;
```

```
DELIMITER //
```

```
CREATE TRIGGER trg_EventApproved
```

```
AFTER UPDATE ON EVENT
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF NEW.ApprovalStatus = 'Approved' AND OLD.ApprovalStatus != 'Approved' THEN
```

```
INSERT INTO EMAIL_LOG (EventID, Message)
```

```
VALUES (NEW.EventID, 'Email sent to Maintenance, Office Services, Security: Event Approved.');
```

```
END IF;
```

```
END //
```

```
DELIMITER ;
```

Part F:

Start of part F

Test Case 1:

VERIFY DATA POPULATION

Expected Result: We should see rows of event data.

--test1 start-

USE UniversityEventsDB;

SELECT * FROM EVENT LIMIT 10;

--test1 ends—

Test 1 result:

	EventID	EventName	StartDateTime	EndDateTime	ApprovalStatus	RejectionJustification	VenueID	DeptCode
▶	1	Varsity Soccer Game	2023-11-01 14:00:00	2023-11-01 16:00:00	Approved	NULL	101	BIO
	2	Swim Meet	2023-11-02 09:00:00	2023-11-02 12:00:00	Approved	NULL	104	BIO
	3	Tennis Finals	2023-11-03 10:00:00	2023-11-03 13:00:00	Approved	NULL	103	MATH
	4	Basketball Practice	2023-11-04 18:00:00	2023-11-04 20:00:00	Rejected	Conflicting Schedule	102	CS
	5	Charity Run	2023-11-05 08:00:00	2023-11-05 11:00:00	Approved	NULL	401	PHYS
	6	Varsity Football	2023-11-06 19:00:00	2023-11-06 21:00:00	Pending	NULL	101	BIO

Test Case 2:

VERIFY THE VIEW

Prove 'ApprovedEventDetails' correctly joins 4 tables and hides pending events.

Expected Result: We should ONLY see events with 'Approved' status.

--test2 starts--

SELECT * FROM ApprovedEventDetails;

--test2 ends—

Test 2 result:

	EventName	StartDateTime	Location	Organizer	DeptName
▶	Varsity Soccer Game	2023-11-01 14:00:00	North Field	Dr. Alice Smith	Biology
	Swim Meet	2023-11-02 09:00:00	Swimming Pool	Prof. Bob Jones	Biology
	Tennis Finals	2023-11-03 10:00:00	Tennis Court A	Kevin Kid	Mathematics
	Charity Run	2023-11-05 08:00:00	Central Plaza	Frank Admin	Physics
	Welcome Picnic	2023-12-01 12:00:00	Library Lawn	Mike Major	English Literature
	Movie Night	2023-12-02 19:00:00	Science Hall A	Nancy New	Computer Science

ApprovedEventDetails 8 x

TEST Case 3:

VERIFY THE TRIGGER (Req 8)

Prove that approving an event automatically logs an email.

--test3 starts--

-- Check the log:

```
SELECT * FROM EMAIL_LOG;
```

-- Change an event status from 'Pending' to 'Approved':

-- Event ID 2 is 'Swim Meet' which was Pending

```
UPDATE EVENT
```

```
SET ApprovalStatus = 'Approved'
```

```
WHERE EventID = 2;
```

-- Check the log again.

-- Expected Result: A NEW row should appear for EventID 2.

```
SELECT * FROM EMAIL_LOG;
```

--test3 ends--

First Check log result:

Result Grid				
	LogID	EventID	Message	SentDate
*	NULL	NULL	NULL	NULL

Second check log result, after the update:

Result Grid				
	LogID	EventID	Message	SentDate
▶	1	2	Email sent to Maintenance, Office Services, Sec...	2025-12-12 19:21:38
*	NULL	NULL	NULL	NULL

TEST Case 4:

VERIFY TIME CONSTRAINTS (Req 9)

Prove that the database REJECTS invalid data (Negative Testing).

Test A: Try to insert an event longer than 3 days.

Expected Result: Error Code "Check constraint 'Chk_Duration' violated"

```
INSERT INTO EVENT (EventName, StartDateTime, EndDateTime, VenueID, DeptCode, OrganizerPersonID)
```

```
VALUES ('Long Festival', '2023-12-01 10:00:00', '2023-12-06 10:00:00', 101, 'CS', 1);
```

Test A error result:

✖	236	19:29:37	INSERT INTO EVENT (EventName, StartDateTime, EndDateTime, VenueID, Dept...	Error Code: 3819. Check constraint 'Chk_Duration' is violated.	0.000 sec
---	-----	----------	--	--	-----------

Test B: Try to insert an event starting before 8 AM.

Expected Result: Error Code "Check constraint 'Chk_StartTime' violated"

```
INSERT INTO EVENT (EventName, StartDateTime, EndDateTime, VenueID, DeptCode, OrganizerPersonID)
```

```
VALUES ('Early Riser', '2023-12-01 05:00:00', '2023-12-01 07:00:00', 101, 'CS', 1);
```

Test B error result:

✖	237	19:30:17	INSERT INTO EVENT (EventName, StartDateTime, EndDateTime, VenueID, Dept...	Error Code: 3819. Check constraint 'Chk_StartTime' is violated.	0.000 sec
---	-----	----------	--	---	-----------

TEST Case 5:

VERIFY WEAK ENTITY & CASCADE DELETE

Prove that deleting an Event automatically deletes its Sub-Events.

Verify sub-events exist for Event 16

```
SELECT * FROM SUB_EVENT WHERE EventID = 16;
```

Result:

Result Grid			Filter Rows:			Export/Import: 		Wrap Cell Content: 
	EventID	SubEventID	SubEventName	StartTime	AllocatedTime	InChargePersonID		
▶	16	1	Introduction	2023-09-01 14:00:00	15 mins	11		
	16	2	Main Talk	2023-09-01 14:15:00	60 mins	11		
	16	3	Q&A Session	2023-09-01 15:15:00	30 mins	12		
	16	4	Networking	2023-09-01 15:45:00	15 mins	13		
	16	5	Closing	2023-09-01 16:00:00	10 mins	14		
*	NULL	NULL	NULL	NULL	NULL	NULL		

Delete the main Event 16

```
DELETE FROM EVENT WHERE EventID = 16;
```

Check sub-events again.

Expected Result: Empty result set.

```
SELECT * FROM SUB_EVENT WHERE EventID = 16;
```

Result:

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	EventID	SubEventID	SubEventName	StartTime	AllocatedTime	InChargePersonID
*	NULL	NULL	NULL	NULL	NULL	NULL

End of Part F